Clone Graph (medium)

Problem Statement

Given a reference of a node in a connected undirected graph, return a deep copy (clone) of the graph. Each node in the graph contains a value (`int`) and a list (`List[Node]`) of its neighbors.

Example 1:

Input:

```
1--2
|  |
4--3
```

Expected Output:

```
1--2
|  |
4--3
```

Explanation: The graph has four nodes with the following connections:

- Node `1` is connected to nodes `2` and `4`.
- Node `2` is connected to nodes `1` and `3`.
- Node `3` is connected to nodes `2` and `4`.
- Node `4` is connected to nodes `1` and `3`.

Example 2:

Input:

```
   1--2
  /    \
 5      3
        |
        4
```

Expected Output:

```
   1--2
  /    \
 5      3
        |
        4
```

Explanation: The graph consists of five nodes with these connections:

- Node `1` is connected to nodes `2` and `5`.
- Node `2` is connected to nodes `1` and `3`.
- Node `3` is connected to nodes `2` and `4`.
- Node `4` is connected to node `3`.
- Node `5` is connected to node `1`.

Example 3:

Input:

```
   1--2
  /    \
 4      3
  \    /
   5--6
```

Expected Output:

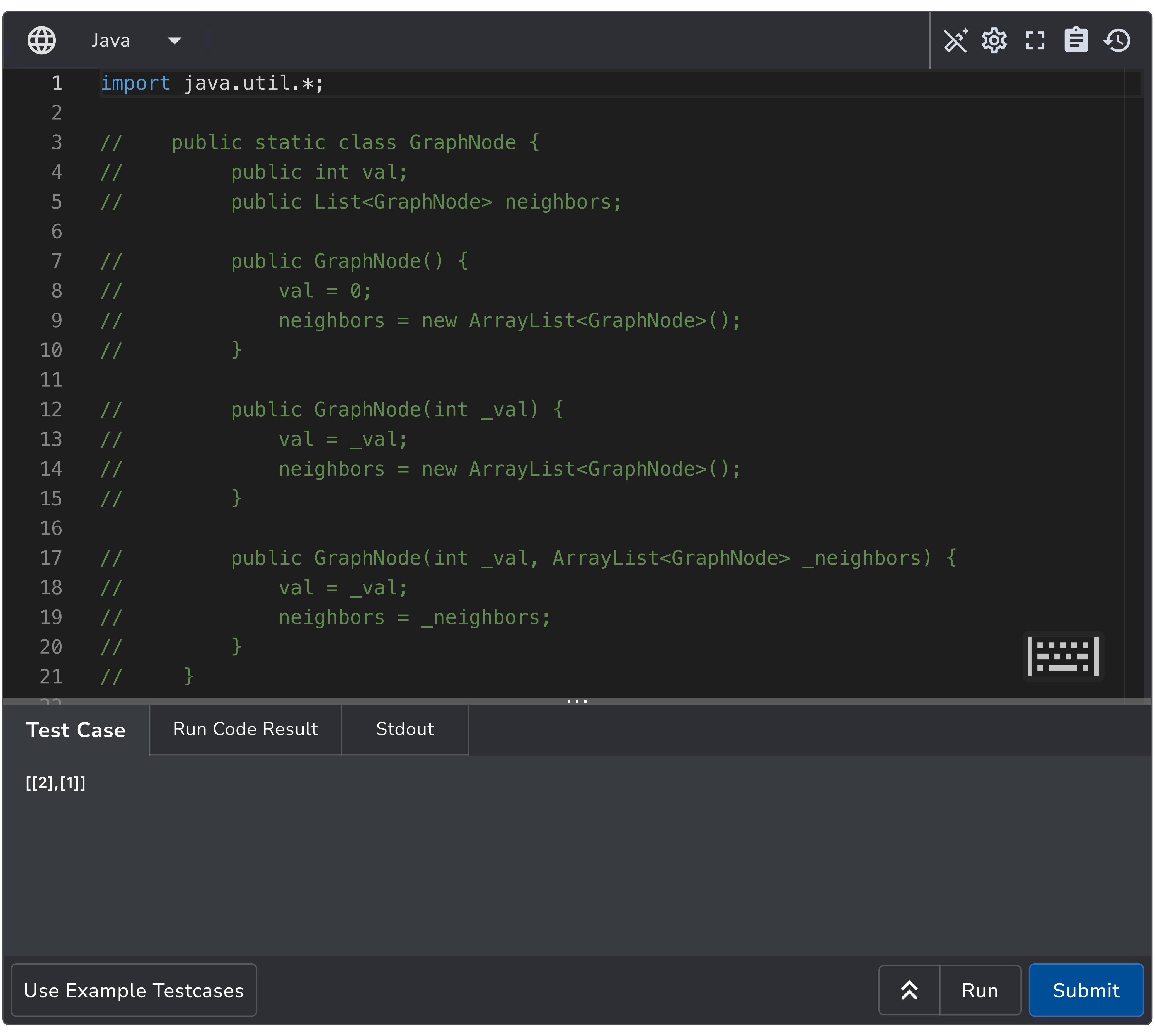
```
   1--2
  /    \
 4      3
  \    /
   5--6
```

Explanation: The graph has six nodes with the following connections:

- Node `1` is connected to nodes `2` and `4`.
- Node `2` is connected to nodes `1` and `3`.
- Node `3` is connected to nodes `2` and `6`.
- Node `4` is connected to nodes `1` and `5`.
- Node `5` is connected to nodes `4` and `6`.
- Node `6` is connected to nodes `3` and `5`.

Try it yourself

Try solving this question here:



```java
import java.util.*;

//    public static class GraphNode {
//        public int val;
//        public List<GraphNode> neighbors;

//        public GraphNode() {
//            val = 0;
//            neighbors = new ArrayList<GraphNode>();
//        }

//        public GraphNode(int _val) {
//            val = _val;
//            neighbors = new ArrayList<GraphNode>();
//        }

//        public GraphNode(int _val, ArrayList<GraphNode> _neighbors) {
//            val = _val;
//            neighbors = _neighbors;
//        }
//     }
```

```
[[2],[1]]
```

← Previous
Solution: Construct Binary Tree From Preorder And Inorder Traversal

Next →
Solution: Clone Graph

☐ Mark as Completed